★ Deploy shop-api on a VPS by hand

Level 1 capstone ~8-10 hours

1 Goal and outcome

Take a blank Ubuntu server and turn it, **by hand and without Docker**, into a small production-style host that serves the `shop-api` sample application at `http://<server-ip>/` (or `https://example.com` if you own a domain):

- **Ubuntu 24.04 LTS** VPS, reached only by SSH key as an unprivileged admin user, with root and password login disabled.
- **Node.js** API (`shop-api`) running as its own non-login system user, supervised by **systemd**, logging to the **journal**.
- **PostgreSQL** holding the data, reachable only on the loopback interface.
- **Nginx** as the public reverse proxy on port 80, the only application port exposed to the internet.
- A **firewall** (`ufw`) that allows just SSH and HTTP(S), plus log rotation, a verification script and a clean teardown.

Why do it manually first? Everything later in this roadmap — Docker, CI/CD, Terraform, Kubernetes — automates or wraps exactly these steps. If you have done them once with your own hands, you know what the tools are doing and how to debug them at 3 a.m.

You are done when `curl http://<server-ip>/products` returns JSON from PostgreSQL, `sudo ./verify.sh` prints `ALL CHECKS PASSED`, and you can explain every file you created.

2 Architecture

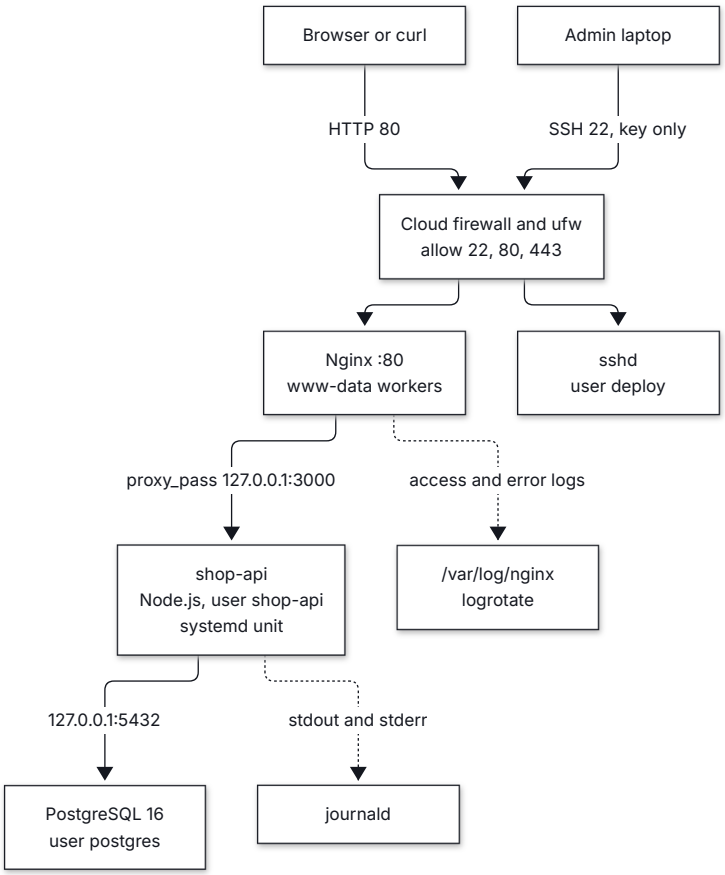


Figure 1-1

The request path and the files that matter:

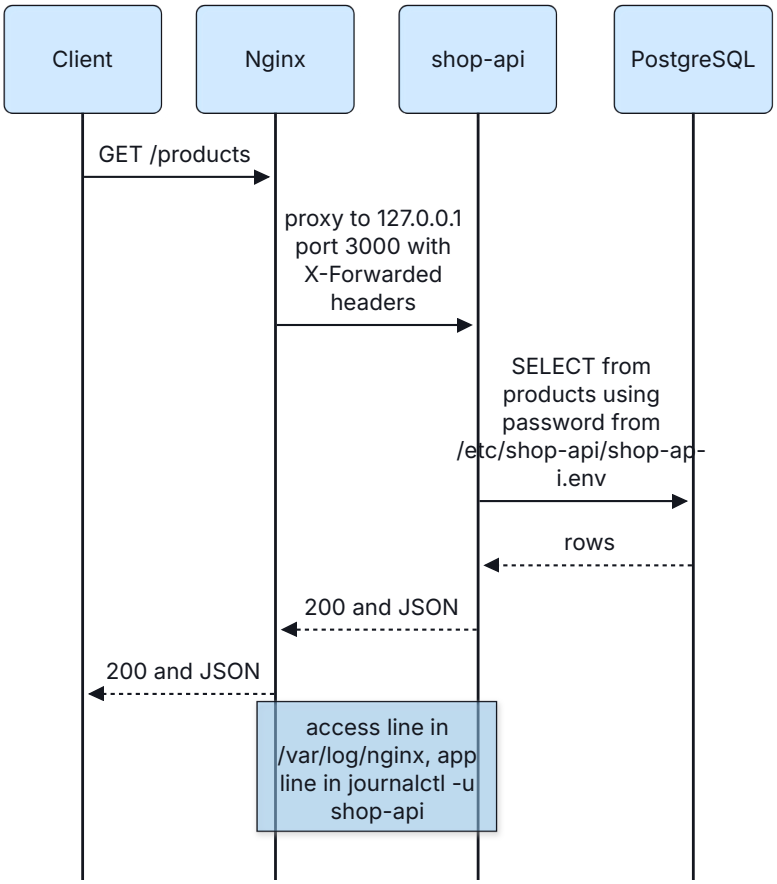


Figure 1-2

Concern	Location on the server
Application code	`/opt/shop-api/` (owned by `deploy`, readable by the service)
Configuration and the DB secret	`/etc/shop-api/shop-api.env` (`root:shop-api`, mode `0640`)
systemd unit	`/etc/systemd/system/shop-api.service`
Nginx site	`/etc/nginx/sites-available/shop-api` (symlinked from `sites-enabled/`)
PostgreSQL data	`/var/lib/postgresql/16/main` (managed by the package)
Logs	`journalctl -u shop-api`, `/var/log/nginx/`, `/var/log/postgresql/`

Running accounts: `deploy` (human admin with `sudo`), `shop-api` (service, `nologin`), `postgres`, and `www-data` (Nginx workers).

3 Prerequisites and cost

- A laptop with a terminal and `ssh` (macOS, Linux, WSL2 or recent Windows).
- **A fresh Ubuntu 24.04 LTS server with a public IP and at least 1 vCPU and 1 GB RAM** from any VPS or cloud provider. Typical small instances cost a few dollars per month and are billed by the hour, so a weekend of work costs very little — **but billing continues until you destroy the server (section 7).** Set a calendar reminder. Free alternative: a local VM (`multipass launch 24.04 --name shop`, VirtualBox, or UTM); you lose only the public-internet parts.
- Optional: a domain name for `example.com`-style access and HTTPS (stretch goals). Documentation addresses below (`203.0.113.10`) are placeholders; use your server's IP.
- Roughly 8-10 hours, ideally two sessions. Read topics 1-13 of this level first, or keep them open: each milestone says which one it uses.

Coverage of the level's topics:

Topic	Where you use it
1 Filesystem hierarchy	M3 layout: `/opt`, `/etc`, `/var`, `/run`
2 Navigating and managing files	all milestones: `mkdir`, `install`, `cp`, `ln -s`, `rm`
3 Viewing and searching text	M8: `less`, `grep`, `journalctl -g`, `tail -f`
4 awk and sed	M5 and M8: edit config with `sed`, summarize the Nginx log with `awk`
5 Network and transfer	M2, M5, M9: `curl`, `ss`, `scp`/`rsync`, `nc`
6 Permissions	M1, M3, M4: `0600`, `0640`, `chown`, the secret file
7 Users, groups, sudo	M1, M3: `deploy`, `shop-api`, `sudo`
8 Processes and signals	M6, M8: PIDs, `SIGTERM` vs `SIGKILL`, restart behavior
9 Daemons and systemd	M6: the unit file
10 System monitoring	M8: `uptime`, `free`, `df`, `top`
11 systemctl and journalctl	M6, M8: service control, logs, rotation
12 SSH keys	M1: keypair, `authorized_keys`, host fingerprint
13 ssh-agent and tunnels	M1, M9: `~/.ssh/config`, tunnel to PostgreSQL

4 Build it step by step

Commands run on **your laptop** are marked `(laptop)`; everything else runs on the **server**. Replace `203.0.113.10` with your server's IP.

M0 — Provision and first login *(topics 5, 12)*

1. Create the server in your provider's console: Ubuntu 24.04 LTS, smallest size, region near you, and **add your SSH public key** in the creation form. If you have no key yet `(laptop)`: `ssh-keygen -t ed25519 -C "you@laptop"` and paste `~/.ssh/id_ed25519.pub`.
2. Connect as root (some providers use `ubuntu` or `debian`-style default users; use whatever the console states, and prefix commands with `sudo` if you are not root). The fingerprint prompt appears only on first contact. Compare it with the one shown in the provider's console or log if available before typing `yes`.

```bash
ssh root@203.0.113.10                    # (laptop)
cat /etc/os-release | head -2            # PRETTY_NAME="Ubuntu 24.04 ... LTS"
uptime; free -h; df -h /                 # baseline: topic 10
sudo apt-get update && sudo apt-get -y upgrade
```

   If the upgrade installs a new kernel it may say a reboot is needed: `sudo reboot`, wait a minute, reconnect.

M1 — Admin user and SSH hardening *(topics 7, 12, 13, 6)*

1. Create an unprivileged admin with `sudo` and your key. (Do not leave it passwordless: `sudo` will ask for this password. Choose a strong one.)

```bash
adduser --gecos "" deploy                # prompts for a password
usermod -aG sudo deploy
install -d -m 700 -o deploy -g deploy /home/deploy/.ssh
cp ~/.ssh/authorized_keys /home/deploy/.ssh/authorized_keys     # if you are root and
  the key came from the console
chown deploy:deploy /home/deploy/.ssh/authorized_keys
chmod 600 /home/deploy/.ssh/authorized_keys
```

2. **In a second terminal** `(laptop)` confirm the new login works *before* changing anything else. Add a convenient alias:

```bash
cat >> ~/.ssh/config <<'EOF'

Host shop
    HostName 203.0.113.10
    User deploy
    IdentityFile ~/.ssh/id_ed25519
    IdentitiesOnly yes
    ServerAliveInterval 30
EOF
ssh shop 'id && sudo -l | head -3'         # uid=1001(deploy) ... groups=...,27(sudo)
```

3. Harden `sshd` with a drop-in file (it is read before cloud-init's `50-*.conf`, and for sshd the first value wins):

```bash
sudo tee /etc/ssh/sshd_config.d/10-hardening.conf > /dev/null <<'EOF'
PermitRootLogin no
PasswordAuthentication no
KbdInteractiveAuthentication no
PubkeyAuthentication yes
MaxAuthTries 3
EOF
sudo sshd -t && sudo systemctl reload ssh      # validate, then apply
sudo sshd -T | grep -iE '^(permitrootlogin|passwordauthentication)'
```

   Expected: `permitrootlogin no`, `passwordauthentication no`. **Keep your existing session open**, and from a new terminal run `ssh shop` (works) and `ssh -o PubkeyAuthentication=no deploy@203.0.113.10` and `ssh root@203.0.113.10` (both refused with `Permission denied`). From now on, work as `deploy` with `sudo`.

M2 — Firewall and baseline *(topics 5, 10)*

Allow SSH **before** enabling the firewall, or you will lock yourself out.

```bash
sudo apt-get install -y ufw curl
sudo ufw default deny incoming
sudo ufw default allow outgoing
sudo ufw allow OpenSSH            # port 22
sudo ufw allow 80/tcp
sudo ufw allow 443/tcp            # for the optional HTTPS stretch goal
sudo ufw enable                   # answer y
sudo ufw status verbose           # Status: active, rules for 22, 80, 443
```

Also check the provider's own cloud firewall or security group (if any) allows the same three ports: it is a second layer, in front of `ufw`. See what already listens: `sudo ss -tlnp`. Right now only `sshd` should be public.

M3 — Service account and directory layout *(topics 1, 6, 7)*

```bash
sudo useradd --system --no-create-home --shell /usr/sbin/nologin shop-api
id shop-api                                                  # low UID, own group
sudo install -d -o deploy -g deploy -m 755 /opt/shop-api # code
sudo install -d -o root -g shop-api -m 750 /etc/shop-api # config directory
ls -ld /opt/shop-api /etc/shop-api
```

Rule of the hierarchy: code in `/opt`, configuration in `/etc`, state in `/var/lib`, logs in `/var/log` (or the journal).

M4 — PostgreSQL *(topics 5, 6, 12)*

```bash
sudo apt-get install -y postgresql          # PostgreSQL 16 on Ubuntu 24.04
systemctl is-active postgresql
sudo ss -tlnp | grep 5432                   # 127.0.0.1:5432 and [::1]:5432 only
```

The package listens on localhost only (`listen_addresses = 'localhost'`) and its `pg_hba.conf` requires password auth over TCP. Leave it that way.

Create the role, the database, the table and the secret file. Run it all in one shell session so `$DB_PASS` stays available:

```bash
```

```
DB_PASS=$(openssl rand -hex 24)             # random password, 48 hex characters
sudo -u postgres psql -v ON_ERROR_STOP=1 <<EOF
CREATE ROLE shop LOGIN PASSWORD '${DB_PASS}';
CREATE DATABASE shopdb OWNER shop;
EOF

PGPASSWORD="$DB_PASS" psql -h 127.0.0.1 -U shop -d shopdb -v ON_ERROR_STOP=1 <<'EOF'
CREATE TABLE IF NOT EXISTS products (
  id          serial PRIMARY KEY,
  name        text NOT NULL,
  price_cents integer NOT NULL CHECK (price_cents ⩾ 0),
  created_at  timestamptz NOT NULL DEFAULT now()
);
INSERT INTO products (name, price_cents) VALUES
  ('Keyboard', 4999), ('Mouse', 1999), ('USB-C cable', 899);
SELECT id, name, price_cents FROM products;
EOF
```

Expected: three rows. Now store the connection settings where only root and the service group can read them:

```
BASH
sudo tee /etc/shop-api/shop-api.env > /dev/null <<EOF
NODE_ENV=production
HOST=127.0.0.1
PORT=3000
DATABASE_URL=postgres://shop:${DB_PASS}@127.0.0.1:5432/shopdb
EOF
sudo chown root:shop-api /etc/shop-api/shop-api.env
sudo chmod 640 /etc/shop-api/shop-api.env
ls -l /etc/shop-api/shop-api.env           # -rw-r----- 1 root shop-api
unset DB_PASS
```

M5 — Node.js and the application *(topics 2, 4, 5)*

1. Install a current Node.js LTS from the NodeSource apt repository (Ubuntu's own `nodejs` package is older). Check nodejs.org for the supported LTS lines; the commands work for any listed major version.

```bash
sudo apt-get install -y ca-certificates curl gnupg
sudo mkdir -p /etc/apt/keyrings
curl -fsSL https://deb.nodesource.com/gpgkey/nodesource-repo.gpg.key | sudo gpg --
  dearmor -o /etc/apt/keyrings/nodesource.gpg
NODE_MAJOR=22
echo "deb [signed-by=/etc/apt/keyrings/nodesource.gpg]
  https://deb.nodesource.com/node_${NODE_MAJOR}.x nodistro main" \
  | sudo tee /etc/apt/sources.list.d/nodesource.list
sudo apt-get update && sudo apt-get install -y nodejs
node -v && npm -v && which node            # /usr/bin/node
```

2. Create the application as `deploy` in `/opt/shop-api`:

```bash
cd /opt/shop-api
cat > package.json <<'EOF'
{
  "name": "shop-api",
  "version": "1.0.0",
  "private": true,
  "main": "server.js",
  "dependencies": { "pg": "^8.11.0" }
}
EOF
cat > server.js <<'EOF'
'use strict';
const http = require('node:http');
const { Pool } = require('pg');

const PORT = Number(process.env.PORT || 3000);
const HOST = process.env.HOST || '127.0.0.1';
const pool = new Pool({ connectionString: process.env.DATABASE_URL, max: 5 });
// An idle connection can be dropped (for example a database restart). Without this
  handler Node would exit.
pool.on('error', (err) => log('error', 'idle db client error', { error: err.message
  }));

function log(level, msg, extra = {}) {
  console.log(JSON.stringify({ time: new Date().toISOString(), level, msg, ...extra
  }));
}
function send(res, status, body) {
  res.writeHead(status, { 'Content-Type': 'application/json' });
  res.end(JSON.stringify(body));
}

const server = http.createServer(async (req, res) => {
  const started = Date.now();
  res.on('finish', () => log('info', 'request',
```

```
    { method: req.method, url: req.url, status: res.statusCode, ms: Date.now() -
  started }));
  try {
    const url = new URL(req.url, 'http://localhost');
    if (req.method === 'GET' && url.pathname === '/health') {
      await pool.query('SELECT 1');
      return send(res, 200, { status: 'ok', db: 'up' });
    }
    if (req.method === 'GET' && url.pathname === '/products') {
      const { rows } = await pool.query('SELECT id, name, price_cents FROM products
  ORDER BY id');
      return send(res, 200, rows);
    }
    const m = url.pathname.match(/^\/products\/(\d+)$/);
    if (req.method === 'GET' && m) {
      const { rows } = await pool.query('SELECT id, name, price_cents FROM products
  WHERE id = $1', [m[1]]);
      return rows.length ? send(res, 200, rows[0]) : send(res, 404, { error: 'not
  found' });
    }
    return send(res, 404, { error: 'not found' });
  } catch (err) {
    log('error', 'handler failed', { error: err.message });
    return send(res, 503, { status: 'error', error: 'database unavailable' });
  }
});

server.listen(PORT, HOST, () => log('info', 'shop-api listening', { host: HOST, port:
  PORT }));

function shutdown(signal) {            // graceful stop on SIGTERM: stop accepting,
  drain, close the pool
  log('info', 'shutting down', { signal });
  server.close(async () => { await pool.end(); process.exit(0); });
  setTimeout(() => process.exit(1), 10000).unref();
}
process.on('SIGTERM', () => shutdown('SIGTERM'));
process.on('SIGINT', () => shutdown('SIGINT'));
EOF
npm install --omit=dev --no-audit --no-fund
ls                                          # node_modules  package-lock.json
  package.json  server.js
```

3. Smoke-test it by hand, as the service user, with the real environment file. This proves permissions before systemd is involved:

```bash
sudo -u shop-api bash -c 'set -a; . /etc/shop-api/shop-api.env; set +a; cd /opt/shop-api && exec node server.js' &
sleep 1
curl -s http://127.0.0.1:3000/health ; echo        # {"status":"ok","db":"up"}
curl -s http://127.0.0.1:3000/products ; echo      # the three products as JSON
kill %1                                            # SIGTERM: you should see "shutting down" in the output
```

If this fails with `permission denied` check the modes from M3 and M4. For later deployments, push new code with `rsync -av --exclude node_modules ./ shop:/opt/shop-api/` `(laptop)`.

M6 — Run it under systemd *(topics 8, 9, 11)*

```bash
sudo tee /etc/systemd/system/shop-api.service > /dev/null <<'EOF'
[Unit]
Description=shop-api (Node.js)
After=network-online.target postgresql.service
Wants=network-online.target postgresql.service

[Service]
Type=simple
User=shop-api
Group=shop-api
WorkingDirectory=/opt/shop-api
EnvironmentFile=/etc/shop-api/shop-api.env
ExecStart=/usr/bin/node /opt/shop-api/server.js
Restart=on-failure
RestartSec=3
TimeoutStopSec=20
NoNewPrivileges=true
ProtectSystem=strict
ProtectHome=true
PrivateTmp=true

[Install]
WantedBy=multi-user.target
EOF
sudo systemd-analyze verify /etc/systemd/system/shop-api.service
sudo systemctl daemon-reload
sudo systemctl enable --now shop-api
systemctl status shop-api --no-pager        # Active: active (running)
curl -s http://127.0.0.1:3000/health ; echo
journalctl -u shop-api -n 5 --no-pager      # JSON lines: "shop-api listening"
```

Prove the supervision: `sudo kill -9 "$(systemctl show -p MainPID --value shop-api)"`, wait four seconds, and `systemctl show -p MainPID,NRestarts shop-api` shows a new PID and `NRestarts=1`. Then `sudo systemctl stop shop-api` and read the `shutting down` line from `SIGTERM` in the journal. Start it again.

M7 — Nginx reverse proxy *(topics 2, 5, 4)*

```bash
sudo apt-get install -y nginx
sudo tee /etc/nginx/sites-available/shop-api > /dev/null <<'EOF'
upstream shop_api {
    server 127.0.0.1:3000;
    keepalive 16;
}

server {
    listen 80 default_server;
    listen [::]:80 default_server;
    server_name example.com www.example.com _;

    access_log /var/log/nginx/shop-api.access.log;
    error_log  /var/log/nginx/shop-api.error.log warn;
    client_max_body_size 1m;

    location / {
        proxy_pass http://shop_api;
        proxy_http_version 1.1;
        proxy_set_header Connection "";
        proxy_set_header Host              $host;
        proxy_set_header X-Real-IP         $remote_addr;
        proxy_set_header X-Forwarded-For   $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
        proxy_connect_timeout 5s;
        proxy_read_timeout    30s;
    }
}
EOF
sudo ln -s /etc/nginx/sites-available/shop-api /etc/nginx/sites-enabled/shop-api
sudo rm /etc/nginx/sites-enabled/default       # the welcome page would otherwise also
  claim port 80
sudo nginx -t                                   # syntax is ok / test is successful
sudo systemctl enable --now nginx
sudo systemctl reload nginx
curl -s http://127.0.0.1/products ; echo
```

From your laptop `(laptop)`: `curl -s http://203.0.113.10/health` returns `{"status":"ok","db":"up"}`, and `curl --max-time 5 http://203.0.113.10:3000/health` must **fail** (port 3000 is bound to loopback and blocked by the firewall).

M8 — Logs, rotation and monitoring *(topics 3, 4, 10, 11)*

1. Make the journal persistent and bounded:

```bash
sudo mkdir -p /etc/systemd/journald.conf.d
printf '[Journal]\nStorage=persistent\nSystemMaxUse=500M\nMaxRetentionSec=1month\n' |
  sudo tee /etc/systemd/journald.conf.d/50-size.conf
sudo systemctl restart systemd-journald
journalctl --disk-usage
```

2. Nginx logs are rotated by the package rule `/etc/logrotate.d/nginx`; confirm and dry-run it:

```bash
cat /etc/logrotate.d/nginx | head -12
sudo logrotate -d /etc/logrotate.d/nginx 2>&1 | head -8
systemctl list-timers logrotate.timer --no-pager
```

3. Generate traffic and read the logs with the text tools:

```bash
for i in $(seq 20); do curl -s -o /dev/null http://127.0.0.1/products; done; curl -s -
  o /dev/null http://127.0.0.1/nope
sudo awk '{print $9}' /var/log/nginx/shop-api.access.log | sort | uniq -c | sort -rn
  # counts per HTTP status
sudo awk '{print $7}' /var/log/nginx/shop-api.access.log | sort | uniq -c | sort -rn |
  head -3
journalctl -u shop-api -g '"status":404' --no-pager | tail -2
journalctl -u shop-api -p err --since "1 hour ago" --no-pager
```

4. A one-minute health glance, the same one you would run in an incident:

```bash
uptime; nproc; free -h; df -h /; ps -eo pid,user,%cpu,%mem,cmd --sort=-%mem | head -5
```

M9 — Verify, then tunnel *(topics 5, 10, 13)*

Create `/home/deploy/verify.sh` (it needs `sudo` because it reads `ufw` and `sshd` settings):

```bash
cat > ~/verify.sh <<'EOF'
#!/usr/bin/env bash
# verify.sh - checks the manual shop-api stack. Exit 0 only if everything passes.
set -u
fail=0
check() {                       # usage: check "description" command args...
```

```
  local desc=$1; shift
  if "$@" >/dev/null 2>&1; then printf 'PASS  %s\n' "$desc"; else printf 'FAIL  %s\n'
  "$desc"; fail=1; fi
}
not_public() {                  # true when nothing listens on 0.0.0.0 or [::] for the
  given port
  ! ss -H -tln "sport = :$1" | awk '{print $4}' | grep -Eq '^(0\.0\.0\.0|\*|\[::\]):'
}
runs_as() {                     # runs_as unit user
  local pid; pid=$(systemctl show -p MainPID --value "$1")
  [ "$pid" -ne 0 ] && [ "$(ps -o user= -p "$pid")" = "$2" ]
}

check "nginx active"                    systemctl is-active --quiet nginx
check "postgresql active"               systemctl is-active --quiet postgresql
check "shop-api active"                 systemctl is-active --quiet shop-api
check "shop-api enabled at boot"        systemctl is-enabled --quiet shop-api
check "shop-api runs as shop-api"       runs_as shop-api shop-api
check "API /health through nginx"       curl -fsS --max-time 5 -H 'Host: example.com'
  http://127.0.0.1/health
check "API /products returns data"      bash -c "curl -fsS --max-time 5
  http://127.0.0.1/products | grep -q Keyboard"
check "port 3000 not public"            not_public 3000
check "port 5432 not public"            not_public 5432
check "ufw active"                      bash -c "ufw status | grep -q 'Status: active'"
check "ssh password login disabled"     bash -c "sshd -T | grep -qi
  '^passwordauthentication no'"
check "ssh root login disabled"         bash -c "sshd -T | grep -qi '^permitrootlogin no'"
check "env file mode is 640"            bash -c "[ \"\$(stat -c %a /etc/shop-api/shop-
  api.env)\" = 640 ]"
check "root disk below 80 percent"      bash -c "[ \"\$(df --output=pcent / | tail -1 | tr
  -dc 0-9)\" -lt 80 ]"

[ "$fail" = 0 ] && echo "ALL CHECKS PASSED" || echo "SOME CHECKS FAILED"
exit "$fail"
EOF
chmod +x ~/verify.sh && sudo ~/verify.sh
```

Finally, reach PostgreSQL from your laptop **without** opening its port, using an SSH tunnel `(laptop)`:

BASH

```
ssh -fN -L 5433:127.0.0.1:5432 shop
psql -h 127.0.0.1 -p 5433 -U shop -d shopdb -c 'SELECT count(*) FROM products;'   # asks
  for the DB password from /etc/shop-api/shop-api.env
kill "$(pgrep -f 'ssh -fN -L 5433')"
```

5 Tasks

- [ ] **T1 — Build the foundation (M0-M2)** *(≈90 min, beginner)* — exercises topics 5, 7, 12
  - **Goal:** a server you can only enter by key, as `deploy`, behind a firewall.
  - **Steps:**
    1. Provision the VPS, create `deploy`, install your key, and add the `Host shop` entry.
    2. Apply the sshd drop-in, validate with `sshd -t`, reload, and test from a second terminal.
    3. Enable `ufw` with SSH, 80 and 443.
  - **Done when:** `ssh shop` works, `ssh root@<ip>` and password logins are refused, and `sudo ufw status verbose` shows exactly three allowed ports.
- [ ] **T2 — Service account and layout (M3)** *(≈30 min, beginner)* — exercises topics 1, 6, 7
  - **Goal:** put every file where operators expect it, with the least privilege.
  - **Steps:**
    1. Create `shop-api` as a `nologin` system user and the directories `/opt/shop-api` and `/etc/shop-api`.
    2. Try `sudo -u shop-api bash` (it refuses) and `sudo -u shop-api ls /etc/shop-api` (works), then `sudo -u shop-api touch /etc/shop-api/x` (fails).
  - **Done when:** you can state the owner, group and mode of each directory and explain why the service cannot write to its own code or config.
- [ ] **T3 — Database and app (M4-M5)** *(≈90 min, intermediate)* — exercises topics 2, 5, 6
  - **Goal:** the API answers from PostgreSQL when run by hand as `shop-api`.
  - **Steps:**
    1. Install PostgreSQL, create the role, database, table and seed rows; write the env file with mode `0640`.
    2. Install Node.js, create the app, run it manually as `shop-api`, and call `/health` and `/products`.
    3. Show `ss -tlnp` has PostgreSQL and the app listening only on loopback.
  - **Done when:** `curl` returns the three products, `ls -l` shows `-rw-r----- root shop-api` on the env file, and `sudo -u nobody cat /etc/shop-api/shop-api.env` is denied.
- [ ] **T4 — Systemd and Nginx (M6-M7)** *(≈90 min, intermediate)* — exercises topics 8, 9, 11
  - **Goal:** a supervised service behind a reverse proxy.
  - **Steps:**

1. Install the unit, enable it, kill the process with `SIGKILL`, and watch it return; then stop it with `SIGTERM` and find the graceful-shutdown line in the journal.
2. Install the Nginx site, run `nginx -t`, `reload`, and call the API from your laptop on port 80.
3. Run `sudo systemctl reboot` and verify everything is back without any manual start.

- **Done when:** after the reboot `curl http://<ip>/health` works, `systemctl is-enabled shop-api nginx postgresql` prints `enabled` three times, and `NRestarts` shows your kill.

☐ **T5 — Break it and fix it (M6-M8)** *(≈60 min, advanced)* — exercises topics 3, 8, 10, 11

- **Goal:** diagnose five failures using only logs and status commands.
- **Steps:** for each fault below, observe the symptom (`curl` status or `systemctl status`), find the evidence (journal, Nginx error log, `ss`, `ps`), fix it, and write down the line that proved the cause:
  1. `sudo systemctl stop postgresql` (expect `503` from `/health`; start it again and show the API recovers without restarting it).
  2. `sudo chmod 600 /etc/shop-api/shop-api.env` (the service cannot read it after a restart).
  3. `sudo systemctl stop shop-api` (Nginx answers `502 Bad Gateway`; find `connect() failed` in `/var/log/nginx/shop-api.error.log`).
  4. Change the Nginx `proxy_pass` port to `3001`, run `nginx -t` (passes), reload, and diagnose the resulting `502`.
  5. `sudo ufw deny 80/tcp` (laptop requests time out while local `curl 127.0.0.1` still works).
- **Done when:** all five are repaired, `sudo ./verify.sh` is green again, and your notes name the evidence for each.

☐ **T6 — Tunnel and observe (M8-M9)** *(≈45 min, intermediate)* — exercises topics 4, 10, 13

- **Goal:** use the admin tools you would use in production.
- **Steps:**
  1. Open the PostgreSQL tunnel from your laptop and run a query through it; confirm `nc -zv <ip> 5432` from the laptop fails.
  2. Use `awk` to compute the slowest request in `journalctl -u shop-api -o cat` (the JSON `ms` field; `grep -o '"ms":[0-9]*' | cut -d: -f2 | sort -n | tail -1` is enough), and the 5xx count in the Nginx log.

3. Record `uptime`, `free -h`, `df -h /` before and after sending 500 requests with a shell loop.

- **Done when:** the tunneled query returns a row count, the direct connection is refused, and you have the before/after numbers.

- [ ] **T7 — Add a nightly backup with a systemd timer** *(≈60 min, advanced)* — exercises topics 6, 9, 11
  - **Goal:** schedule a `pg_dump` the systemd way and prove a restore.
  - **Steps:**
    1. Create `/var/backups/shop-api` (`postgres`-owned, `0700`) and a `shop-backup.service` (`User=postgres`, `ExecStart=/bin/sh -c 'pg_dump -Fc shopdb > /var/backups/shop-api/shopdb-$(date +%%F).dump'`) with a `shop-backup.timer` (`OnCalendar=*-*-* 03:00:00`, `Persistent=true`). (Running as the `postgres` OS user uses peer authentication, so no password is needed. In a unit file `%%` is a literal `%`.)
    2. Run the service once with `systemctl start`, read the journal, then restore it with `sudo -u postgres createdb shopdb_restore` and `sudo -u postgres pg_restore --no-owner -d shopdb_restore <dump file>`, and compare row counts.
  - **Done when:** `systemctl list-timers shop-backup.timer` shows the next run, a dump file exists, and the restored database has the same three rows.

- [ ] **T8 — Prove it: rebuild without the guide** *(≈2-3 hours, advanced)*
  - **Goal:** show you own the whole stack.
  - **Steps:**
    1. Destroy the server (or create a second one). Without opening this page, rebuild it using only your own notes and the `man` pages, in the same order.
    2. Turn your notes into a single idempotent `bootstrap.sh` (or an Ansible-style checklist) and run it on a clean machine.
    3. Explain aloud, to a teammate or in writing, what each of the 13 topics did for you in this build.
  - **Done when:** `sudo ./verify.sh` prints `ALL CHECKS PASSED` on the rebuilt server and your bootstrap script runs twice without errors.

6 Verification checklist

Run each command on the server (or laptop where marked) and tick the box when the result matches.

- [ ] `sudo ./verify.sh` ends with `ALL CHECKS PASSED`.

- [ ] `(laptop)` `curl -s http://<ip>/health` returns `{"status":"ok","db":"up"}`; `curl -s http://<ip>/products` returns three products.
- [ ] `(laptop)` `ssh root@<ip>` and password login are both refused; `ssh shop` works.
- [ ] `sudo ss -tlnp` shows only `sshd` and `nginx` on `0.0.0.0` / `[::]` (ports 22 and 80); ports 3000 and 5432 are on `127.0.0.1` / `[::1]` only.
- [ ] `ps -o user,pid,cmd -C node` shows `shop-api`, not `root` or `deploy`.
- [ ] `ls -l /etc/shop-api/shop-api.env` shows `-rw-r----- root shop-api`; `sudo -u nobody cat` of it fails.
- [ ] `systemctl is-enabled shop-api nginx postgresql` prints `enabled` for all three, and the stack survives `sudo reboot`.
- [ ] After `sudo kill -9` of the Node process, `systemctl show -p NRestarts shop-api` increases and the API answers again within seconds.
- [ ] `journalctl -u shop-api -n 20` shows JSON request lines; `journalctl --disk-usage` is bounded by your `SystemMaxUse`.
- [ ] `sudo logrotate -d /etc/logrotate.d/nginx` shows the Nginx rule would run; `df -h /` is below 80 percent.
- [ ] `sudo ufw status verbose` lists only 22, 80 and 443 (443 only if you kept it).

7 Teardown

Destroy the server so billing stops. This is the step people forget.

1. Optional: save anything you want to keep `(laptop)`: `scp shop:~/verify.sh .` and your notes.
2. In the provider's console, **delete (destroy) the server**, and delete any attached volumes, snapshots, reserved/floating IPs and DNS records you created. Stopping the server is not enough on most providers: stopped servers can still be billed.
3. Remove local traces `(laptop)`: delete the `Host shop` block from `~/.ssh/config`, remove the stale host key with `ssh-keygen -R 203.0.113.10`, and `ssh-add -d` any key you used only for this exercise.
4. Next day: check the provider's billing page to confirm no resources remain.

If you used a local VM, `multipass delete shop && multipass purge` (or delete the VM in your hypervisor). If you want to keep the server but undo the project:

```bash
sudo systemctl disable --now shop-api nginx
sudo rm /etc/systemd/system/shop-api.service /etc/nginx/sites-enabled/shop-api
  /etc/nginx/sites-available/shop-api
sudo rm -r /etc/shop-api /opt/shop-api
sudo -u postgres psql -c 'DROP DATABASE shopdb;' -c 'DROP ROLE shop;'
sudo userdel shop-api
sudo systemctl daemon-reload
```

8 Stretch goals and what to learn next

- **HTTPS with Let's Encrypt:** point a DNS `A` record for your domain at the server, set `server_name` to it, then `sudo apt-get install -y certbot python3-certbot-nginx && sudo certbot --nginx -d example.com`. Verify the renewal timer with `systemctl list-timers | grep certbot`. Background: TCP Connections and TLS Handshakes §2.6.
- **More hardening:** `fail2ban` or `ufw limit OpenSSH`, unattended security upgrades (`unattended-upgrades`), `AllowUsers deploy`, Nginx security headers. Next topic: Linux Security and SSH Hardening §11.1.
- **Zero-downtime deploys:** run two instances on ports 3000 and 3001 and add both to the `upstream`; reload one at a time. See Deployment Strategies §0.7 and Rolling Deployments §5.10.
- **Automate it:** turn this runbook into a script, then a configuration-management role, then Terraform plus cloud-init. See Configuration Management §0.4 and Infrastructure as Code §0.3.
- **Containerize it:** the same API in a container, with `docker compose` for Nginx, the app and PostgreSQL: Docker Compose §4.8.
- **Ship it from Git:** a pipeline that deploys on push, via SSH, to this server: GitHub Actions Workflows §3.11 and CI/CD §0.2.
- **Back it up properly:** off-site copies and tested restores: Backups and Point-in-Time Recovery §13.5.

Next level: Level 2 — Networking §2.1 explains what you just configured at the packet level: ports, firewalls, DNS, reverse proxies and what really happens when a browser visits your server.